Extracting a crossing in time $O(nm)$

MohammadTaghi Hajiaghayi Mahdi JafariRaviz

November 2026

Abstract

Let C be a cycle in a finite simple graph G with n vertices and m edges. We give an algorithm that either finds two vertex-disjoint C-paths whose ends alternate around C, or correctly reports that there are none. It runs in time $O(nm)$. The algorithm deletes parts of the graph behind separators of order at most three, contracts edges, and lifts the crossing it finds in the small graph back to G. Besides graph searches and bounded flows, it uses one more primitive: a Boolean linear-time planarity test. In the Lean formalization of the Graph Minors series, this algorithm replaces the two paths algorithm of Shiloach that Graph Minors XIII uses in (8.1).

Introduction

All graphs in this note are finite and simple. For a set U of vertices of a graph K, we write $N_K(U)$ for the set of vertices outside U that have a neighbor in U. For a set X of vertices, $K[X]$ is the subgraph induced by X.

Let C be a cycle of a graph G. A *C-path* is a path with at least one edge whose two ends lie on C and whose other vertices do not. A *crossing* of C is a pair of vertex-disjoint C-paths, one with ends a, b and the other with ends c, d, such that a, c, b, d occur in this cyclic order around C. The cycle C is *crossed* if it has a crossing, and *crossless* otherwise. These are the definitions of Humeau and Pous [HP25, Definition 2.4, page 7].

Theorem. *There is an algorithm that, given a graph G with n vertices and m edges and a cycle C of G, returns a crossing of C if C is crossed, and reports that C is crossless otherwise. Its running time is $O(nm)$.*

Where the theorem is used. Algorithm (8.1) of Graph Minors XIII [GM13, page 93] finds a cross in a society. It builds an auxiliary graph and applies an algorithm of Shiloach [Sh80] to find two vertex-disjoint paths joining s_1, t_1 and s_2, t_2. Shiloach's paper describes the algorithm through a sequence of reductions, and on page 454 it says: "We have not written the solution as an explicit, long, and tedious algorithm." For distinct vertices s_1, s_2, t_1, t_2 of a graph H, two vertex-disjoint paths joining s_1, t_1 and s_2, t_2 are exactly a crossing of the cycle $s_1 s_2 t_1 t_2$ in the graph obtained from H by adding the four edges of this cycle [HP25, Section 1.2, pages 2–3]. So the theorem supplies this step of (8.1). In the Lean formalization of the Graph Minors series, the algorithm of this note is used in place of Shiloach's algorithm (Section 14).

Outline. The algorithm runs once on (G, C). Section 1 states the facts we use from outside. Sections 2 to 4 give a flow primitive, a reversible replacement at a separator of order at most three, and an initial cleaning. Sections 5 and 6 classify a clean instance in linear time. The classification either finds a crossing, or shows that there is none, or finds that the graph is nonplanar, which

implies that the cycle is crossed. In the last case, Sections 7 to 10 shrink the graph by contracting edges, until the classification finds a crossing. The main technical point is that a trial contraction can be checked and repaired with a bounded number of flow searches (Sections 8 and 9). Section 11 lifts the crossing back to G, and Section 12 counts the work. Section 13 lists the assumptions and describes the reference implementation `crossing_algorithm.py` and its tests. Section 14 describes the Lean formalization.

1 The external facts used

Besides Menger's theorem for vertex-disjoint paths and the elementary flow computation of Section 2.1, we use the following facts.

The two paths theorem: the consequence we need

Fact T. *Let C be a cycle of a graph K. If C is crossless and K is nonplanar, then there is a nonempty set $U \subseteq V(K) \setminus V(C)$ with $|N_K(U)| \leq 3$.*

Fact T follows from the generalized two paths theorem of Humeau and Pous [HP25, Theorem 1.3, page 2]. To state it, we recall webs [HP25, Definition 2.1, page 6]. A *rib* with frame C is a planar graph with a plane embedding in which the outer face is bounded by the cycle C, every inner face is a triangle, and every triangle bounds an inner face. A *web* with frame C is obtained from a rib with frame C by adding, for each triangle T of the rib, a clique, possibly with no vertices, whose vertices are adjacent to the three vertices of T and to no other vertex outside the clique. A graph H with a cycle C is *maximally C-crossless* if C is crossless in H but crossed in every graph obtained from H by adding one edge. The generalized two paths theorem says that H is a web with frame C if and only if H is maximally C-crossless. We use only the direction proved in [HP25, Lemma 4.3, page 13]: a maximally C-crossless graph is a web with frame C.

Proof of Fact T. Suppose that C is crossless in K. Add edges between vertices of K, one at a time, as long as C stays crossless. Since K is finite, this stops with a maximally C-crossless graph H on the vertex set of K, and H is a web with frame C. If no clique of the web has a vertex, then H is its rib, which is planar, and so is its subgraph K. Since K is nonplanar, some clique, filling a triangle T, has a vertex. Let U be the vertex set of this clique. Then U is nonempty, and it is disjoint from $V(C)$ because C consists of rib vertices. Since K is a subgraph of H on the same vertex set, $N_K(U) \subseteq N_H(U) = V(T)$, and so $|N_K(U)| \leq 3$. □

This is the only consequence of the two paths theorem that we use.

The Boolean planarity primitive

Fact P. *On an adjacency-list representation of a graph K, a deterministic procedure* Planar(K) *returns whether K is planar in time $O(|V(K)| + |E(K)|)$.*

The procedure need not return an embedding or a Kuratowski subdivision. Hopcroft and Tarjan [HT74] gave the first linear-time planarity test. The companion note [HJ26] proves a Boolean left-right test of this kind in full, and the reference implementation uses that test.

Euler's bound

We also use Euler's bound: a planar graph with $N \geq 3$ vertices has at most $3N - 6$ edges, and a bipartite planar graph with $N \geq 3$ vertices has at most $2N - 4$ edges. In particular, $K_{3,3}$ is

nonplanar: a bipartite planar graph on six vertices has at most eight edges, whereas $K_{3,3}$ has nine.

2 The root condition and the bounded-flow primitive

An *instance* (K, D) consists of a graph K and a cycle D of K. Add a new vertex x, adjacent to every vertex of D, and write $J = K + x$. We call x the *apex*. It is used only in auxiliary computations.

For $k \geq 1$, we say that $\mathcal{R}_k(J, x)$ holds if $J - S$ is connected for every set $S \subseteq V(J) \setminus \{x\}$ with $|S| < k$. All these separators exclude x. We do not claim ordinary k-connectivity, which would also consider separators that contain x.

We call (K, D) *clean* when $\mathcal{R}_4(K + x, x)$ holds. A *four-fan* from a vertex v to D consists of four paths that share only their first vertex v and end at four distinct vertices of D. Because every vertex of D is adjacent to x, Menger's theorem gives

$$(K, D) \text{ is clean} \quad \Longleftrightarrow \quad \text{every vertex } v \notin V(D) \text{ has a four-fan to } D.$$

Indeed, $\mathcal{R}_4(J, x)$ says that no vertex is separated from x by at most three vertices other than itself and x. A vertex of D is adjacent to x. For a vertex $v \notin V(D)$, which is not adjacent to x, Menger's theorem turns this into four paths from v to x that share only v and x. Cutting each of these paths at its first vertex of D gives a four-fan, and extending the paths of a four-fan by their edges to x gives four such paths back.

2.1 Exact bounded-flow computation

Every flow query below has a source z and the sink x, where z is not adjacent to x, and asks for $k \leq 4$ internally vertex-disjoint paths from z to x. We build the split network.

- Replace each vertex v by two nodes v_{in} and v_{out}.
- Give the arc $v_{\text{in}} \to v_{\text{out}}$ capacity 1, except at z and x, where its capacity is $k + 1$.
- For every edge uv, add the arcs $u_{\text{out}} \to v_{\text{in}}$ and $v_{\text{out}} \to u_{\text{in}}$, each of capacity $k + 1$.
- Use the source z_{out} and the sink x_{in}.

Starting with the zero flow, we search the residual graph for a path from the source to the sink and augment one unit along it. We stop after k augmentations or after an unsuccessful search. Thus each query uses at most k residual searches, each on a graph of size $O(|V| + |E|)$. Any augmenting paths work.

If the computation stops below k, its minimum cut consists of arcs $v_{\text{in}} \to v_{\text{out}}$ of capacity 1, because every other arc has capacity $k + 1$, more than the value of the cut. These arcs give a vertex separator of that size between z and x.

2.2 Computing a minimum cut with maximum source side

One later step, Lemma 6, needs a particular minimum separator.

After a failed bounded-flow computation, let W be the set of nodes of the network that *cannot reach the sink in the residual graph*. We compute W by a search from the sink along reversed residual arcs, and take the complement. By a standard property of maximum flows, W is the largest source side of a minimum network cut: it contains the source side of every minimum cut. Let

$$S = \{v : \ v_{\text{in}} \in W, \ v_{\text{out}} \notin W\},$$

let R_x be the component of x in $J - S$, and let

$$U = V(J) \setminus (S \cup R_x).$$

In general, the *open source side* of a vertex separator S' between z and x is the set of vertices outside S' that are not in the component of x in $J - S'$. So U is the open source side of S.

This U contains the open source side of every minimum vertex separator between z and x. To see this, let S' be a minimum vertex separator between z and x, with open source side U'. The nodes $u_{\text{in}}, u_{\text{out}}$ for $u \in U'$, together with the nodes s_{in} for $s \in S'$, form the source side of a network cut of capacity $|S'|$. So this cut is minimum, and its source side is contained in W. Now take a path of J from a vertex of U' to x that avoids S. The corresponding path in the network starts in W and ends at the sink, which is not in W, so it uses an arc that leaves W. That arc is not an arc $v_{\text{in}} \to v_{\text{out}}$ with $v \notin S$, by the definition of S. It is not an arc of capacity $k + 1$ either, since such an arc would make the capacity of the minimum cut larger than its value. This is a contradiction. Hence no vertex of U' is joined to x in $J - S$, and no vertex of U' is in S, because its node u_{out} lies in W. Therefore $U' \subseteq U$.

This fixes the choice of the cut completely. *An arbitrary minimum cut does not suffice for Lemma 6.*

3 A reversible replacement at a small separator

The following operation is used throughout.

Let $A \subseteq V(K) \setminus V(D)$ be a connected set, and let $S = N_K(A)$, with $|S| \leq 3$. The *replacement* deletes A and adds every missing edge of the clique on S. We record the graph before the operation, the set A, and the set S.

Lemma 1 (Replacement preserves crossing existence)**.** *If D is crossed before the replacement, then it is crossed afterward. A crossing afterward can be lifted to a crossing before the replacement with at most one ordinary path search.*

Proof. Forward direction. A crossing path that enters A must use at least two distinct vertices of S, since neither of its ends lies in A. Since $|S| \leq 3$, the two disjoint crossing paths cannot both enter A. For the path that enters A, if there is one, replace the segment between its first and last visits to S by the clique edge between these two vertices. This introduces no new vertices, keeps the ends of the path, and keeps the two paths disjoint.

Reverse direction. If neither returned path uses a newly added edge, we keep both. Otherwise exactly one path uses new edges, because two disjoint paths that both use new clique edges would need four distinct vertices of S. Let p and q be the first and last vertices of S on this path. Here $p \neq q$, since a new edge joins two vertices of S. We search for a path from p to q in $K[A \cup \{p, q\}]$. Such a path exists because A is connected and both p and q have neighbors in A. We replace the whole segment between p and q by this path. Its inner vertices lie in A, which neither returned path used, and they include no vertex of S or D. Thus the new path is simple, the two paths stay disjoint, the ends do not change, and they still alternate around D. □

We also allow deleting several components of $K - S$ at once, all with neighborhood S, provided we record one of them. The recorded component is connected and has a neighbor at every vertex of S. The forward proof is unchanged, and the reverse search uses the recorded component.

Lemma 2 (Replacement preserves connectivity statements on retained vertices)**.** *Let T be a set of retained vertices. Every path of the old graph that avoids T and has retained ends gives a path of the new graph between the same ends that avoids T. This holds with or without the apex.*

Proof. Replace each excursion of the path through a deleted component by the edge of the boundary clique between the vertices where the path enters and leaves the component, or delete the excursion if these two vertices are equal. These boundary vertices lie on the path, so they are not in T. The result is a walk of the new graph that avoids T, and shortcutting repeated vertices gives a path. The apex is not affected, because a deleted component contains no vertex of D. □

Therefore every statement proved earlier that a retained vertex cannot be separated from x by at most a given number of other vertices stays true after the replacement. Lemma 2 concerns ordinary connectivity and does not use the two paths theorem.

4 Initial cleaning

First we discard the components of G that do not contain C, since they cannot contain a C-path. We start with the instance (K, D), where K is the component of G that contains C, and $D = C$.

Then we process the vertices of K that are not on C in any fixed order, skipping vertices that have already been deleted. For each such vertex v that is still present:

1. In $K + x$, run the bounded-flow computation asking for four paths from v to x.
2. If it succeeds, continue with the next vertex.
3. Otherwise, obtain a separator S of size at most three.
4. Find the component A of $K - S$ that contains v.
5. Replace A by the clique on its actual neighborhood $N_K(A)$, using Lemma 1.

The component A contains no cycle vertex. Indeed, every cycle vertex that is still present is adjacent to x, whereas S separates v from x. Since $N_K(A) \subseteq S$, Lemma 1 applies.

Every successful four-path query stays true afterward, by Lemma 2. Every unsuccessful query deletes the vertex it asked about. Thus, after at most n bounded-flow queries, every remaining vertex off the cycle has four paths to x that share only their ends, and the remaining instance is clean by the equivalence of Section 2.

Every replacement deletes at least one vertex and adds at most three edges. We record the replacements for the lifting at the end.

5 A linear procedure for crossings in distinct cycle bridges

We need the following procedure for any instance (K, D):

> Return a crossing whose two paths use distinct D-bridges, or correctly report that no such crossing exists.

A *D-bridge* is either

- a component of $K - V(D)$, together with its edges to D, or
- a single *chord*: an edge that is not an edge of D and has both ends on D.

Different component bridges have disjoint sets of inner vertices, and a chord is its own bridge. Every D-path is a chord or has all its inner vertices in one component of $K - V(D)$, so it uses exactly one bridge. The *attachments* of a component bridge are the vertices of D adjacent to the component, and the attachments of a chord are its two ends. The ends of a D-path are attachments of the bridge it uses.

5.1 Candidate endpoint pairs

We compute the attachment list of every bridge in cyclic order. This takes time $O(n+m)$: compute the components off the cycle, then scan the vertices of D in order and append the identifiers of their incident components, suppressing repeated incidences at the same cycle vertex.

For a bridge with attachment list $t_0, t_1, \ldots, t_{h-1}$, we generate the pairs

$$\{t_i, t_{i+1}\}, \qquad \{t_i, t_{i+2}\},$$

with indices modulo h, and omit pairs whose two elements are equal. Duplicates may be kept. These pairs are the *candidates* of the bridge. There are at most twice as many candidates as attachments.

Lemma 3 (These candidates detect every crossing between two bridges)**.** *Let A and B be the attachment sets of two distinct bridges. Suppose that A contains a_1, a_2 and B contains b_1, b_2 such that the pairs $\{a_1, a_2\}$ and $\{b_1, b_2\}$ alternate around D. Then some candidate of the first bridge alternates with some candidate of the second.*

Proof. There are A-attachments in both open arcs of D determined by b_1 and b_2. Around b_2, choose the nearest A-attachment strictly before it and the nearest one strictly after it. These two attachments lie in the two different open arcs, so they alternate with b_1, b_2. In the cyclic A-attachment list they are consecutive, unless $b_2 \in A$, in which case exactly one attachment, namely b_2, lies between them. Hence they form an A-candidate that alternates with b_1, b_2.

Applying the same argument to B, relative to this A-candidate, gives a B-candidate that alternates with it. □

The candidates at distance two are needed to handle shared attachments.

5.2 Finding differently colored alternating candidates in linear time

Number the vertices of D as $0, \ldots, c-1$ in cyclic order. We represent a candidate as a triple

$$(l, r, \gamma), \qquad l < r,$$

where l and r are the numbers of its two attachments and γ, its *color*, is the identifier of its bridge. Two candidates (l', r', γ') and (l, r, γ) alternate around D exactly when $l' < l < r' < r$ or $l < l' < r < r'$.

Sort the candidates by increasing l, breaking ties by decreasing r. Two stable counting sorts do this in linear time.

We maintain a stack of blocks, each block containing candidates of one color, with the following invariants.

1. Adjacent blocks have different colors.
2. Every right endpoint in a block is at most every right endpoint in the block immediately below it.
3. Before a candidate with left endpoint l is processed, every stored candidate has right endpoint greater than l.

For a new candidate $I = (l, r, \gamma)$:

1. Delete the candidates with right endpoint at most l from the top block. Pop the top block if it becomes empty, and repeat with the new top block.
2. Let B be the first block from the top whose color differs from γ. By invariant 1, this is the top block or the second block.

3. If B exists and its minimum right endpoint r' satisfies $r' < r$, return I and the candidate in B that attains this minimum.
4. Otherwise, append I to the top block if its color is γ, and push a new block containing I if not.

Correctness. In step 3, the returned earlier candidate (l', r', γ') satisfies

$$l' < l < r' < r.$$

Here $l' \le l$ by the sorting order, $l < r'$ by invariant 3, and $r' < r$ by the test. The strict inequality $l' < l$ follows from the tie-breaking order: an earlier candidate with the same left endpoint would have right endpoint at least r. So the two candidates alternate, and they have different colors.

The endpoint-order invariant 2 makes the tested minimum no greater than the right endpoint of any stored candidate of a different color. Thus a failed test excludes every differently colored crossing between the new candidate and a stored one. A candidate that alternates with a later one has right endpoint greater than the left endpoint of the later one, so step 1 has not deleted it when the later one is processed. Therefore the sweep finds a differently colored alternating pair whenever one exists. A failed test also gives exactly the inequality needed to preserve invariant 2 when appending or pushing.

Cleanup needs to inspect only the top block. If that block keeps an active candidate, then by invariant 2 every right endpoint in every lower block is at least its right endpoint, so these candidates are active too.

Linear bound. At any sweep position, one bridge has at most six active candidates. A cut of the cyclic attachment list at the sweep position separates it into two consecutive blocks. At most two pairs at cyclic distance one and four pairs at cyclic distance two cross this cut, counting duplicates. Therefore every stack block has size at most six. Each candidate is inserted once and removed at most once.

The two paths. When an alternating pair of candidates of different colors is found, we make two searches. For a component bridge with component A and selected endpoints p, q, we search in $K[A \cup \{p, q\}]$. For a chord, we use its edge. The two searches may return arbitrary paths: their inner vertices belong to different bridges, and the four selected endpoints are distinct.

We call this whole procedure BridgeCross(K, D). It takes time $O(n + m)$ and, when it succeeds, two path searches.

5.3 Why this procedure is complete on planar graphs

Suppose that a crossing Q, R has the inner vertices of both paths in the same component A of $K - V(D)$. Connect an inner vertex of Q to an inner vertex of R by a path inside A, and shorten it so that its inner vertices avoid $Q \cup R$.

The cycle, the two crossing paths and this connector form a subdivision of $K_{3,3}$. Let q and r be the ends of the connector on Q and R, and let q_1, r_1, q_2, r_2 be the ends of the crossing in cyclic order, with q_1, q_2 the ends of Q. The bipartition of the six branch vertices is

$$\{q_1, q_2, r\}, \qquad \{r_1, r_2, q\}.$$

The nine branch paths are the four arcs of the cycle, the four halves of Q and R, and the connector.

This is impossible in a planar graph, by Euler's bound in Section 1. A chord already forms a separate bridge. Consequently:

Lemma 4. *Every crossing in a planar graph uses distinct cycle bridges. Therefore* BridgeCross *is complete on planar inputs.*

6 Classifying a clean instance

For a clean instance (K, D), we run the following procedure.

```
1: function Classify(K, D)
2:     if BridgeCross(K, D) returns a crossing then
3:         return WITNESS and that crossing
4:     if Planar(K) then
5:         return NO
6:     return YES
```

The answer `YES` means that a crossing exists, but no crossing has been found yet. The classification is exact.

- `WITNESS` is correct by construction.
- `NO` is correct by Lemma 4.
- For `YES`, the graph K is nonplanar. If D were crossless, Fact T would give a nonempty set U of vertices off the cycle with $|N_K(U)| \leq 3$. Then $N_K(U)$ would separate U from x in $K + x$, contradicting cleanliness.

Thus classification takes linear time. The remaining work is to turn a nonplanar `YES` answer into paths by making the graph smaller.

7 Sparsifying without losing a positive answer

We need a sparse graph, so that a bounded number of vertices of bounded degree can be examined.

7.1 Four BFS forests preserve all cuts of order at most three

A breadth-first search scans vertices in the order of a queue. When it scans a vertex s, it marks every unmarked neighbor of s, selects the edge joining it to s, and appends it to the queue. A *BFS spanning forest* of a graph consists of the selected edges, where a new search, with a new root, starts at each vertex that is still unmarked when its turn comes in a fixed order.

Given a graph J, we compute four forests in succession.

1. Take a BFS spanning forest F_1 of J.
2. Delete its edges and take a BFS spanning forest F_2 of the rest.
3. Delete those edges and take F_3.
4. Delete those edges and take F_4.

Let $F = F_1 \cup F_2 \cup F_3 \cup F_4$.

This is the scan-first construction of sparse certificates of Cheriyan, Kao and Thurimella [CKT93]. The following proof gives the exact strong property that we need.

Lemma 5 (Strong certificate)**.** *Let F be the union of k successive BFS forests of J, taken as above. For every set S of vertices with $|S| < k$, the graphs $F - S$ and $J - S$ have exactly the same components.*

Proof. Suppose otherwise. Then there is a partition

$$V(J) = A \dot\cup S \dot\cup B$$

such that F has no edge between A and B, but J has an edge ab with $a \in A$ and $b \in B$. Take A to be one component of $F - S$ that is not a component of $J - S$, and let B contain all other components.

Before the forest F_i is computed, we maintain parts $A_i \supseteq A$ and $B_i \supseteq B$ and a remaining separator S_i, which partition $V(J)$, with the following properties:

- the vertex a is in A_i, and b is in B_i;
- the edge ab is still in the residual graph, that is, it has not been selected by an earlier forest;
- no edge of the current or any later forest joins A_i to B_i.

Initially $A_1 = A$, $B_1 = B$ and $S_1 = S$.

In the residual component that contains ab, the current spanning tree connects a to b. Since it has no edge between A_i and B_i, it contains a vertex of S_i. Let s be the first such vertex scanned.

Suppose that the root of this tree lies in A_i. Then no vertex of B_i was marked before s was scanned. Otherwise, the tree path from the root to that vertex would contain a proper ancestor in S_i, and this ancestor was scanned before that vertex was marked, contradicting the choice of s.

Therefore, when s is scanned, every residual neighbor of s in B_i is unmarked, and the search selects every edge from s to B_i. We move s into A_i. All crossing edges that this move creates have just been selected and removed, so no later forest contains them.

The case where the root lies in B_i is symmetric. If the root lies in S_i, it is s. All its neighbors are unmarked when it is scanned, so all its incident edges are selected, and s can be moved to either side.

Each forest removes one vertex from the separator and preserves the properties. The edge ab is never selected, because it always joins the two parts. After $|S|$ forests the separator is empty. Since $|S| < k$, there is a next forest. It must connect a and b, but it has no edge between the two parts, which now cover $V(J)$. This is a contradiction.

The proof uses simplicity when s is scanned: marking an unmarked neighbor selects its unique edge from s. □

The four passes take time $O(n + m)$. On $n + 1$ vertices, their union has at most $4n$ edges.

7.2 The sparsification step

Suppose that the current clean graph K is nonplanar and has N vertices and M edges.

If $M \leq 16N$, we do nothing. Otherwise:

1. Compute the four-forest certificate of $J = K + x$.
2. Keep its edges that do not contain the apex.
3. Keep all edges of D.
4. Keep any $3N - 5$ edges of K.
5. Delete all other edges of K.

The kept graph has at most

$$4N + N + (3N - 5) = 8N - 5$$

edges.

Adding all edges from the apex to the cycle gives a graph that contains the certificate and is contained in J. By Lemma 5, removing at most three vertices other than x leaves the same components in all three graphs, so the kept graph is still clean. Its $3N - 5$ kept edges of K make it nonplanar by Euler's bound. As in Section 6, Fact T then shows that D is still crossed.

This step only deletes edges, so a crossing found later needs no special lifting across it.

8 The maximum two-cut lemma

This is the central separator lemma.

Lemma 6. *Let B be a graph with a vertex x such that $\mathcal{R}_2(B, x)$ holds. Let $z \neq x$ be a vertex whose minimum vertex separators from x have size two. Choose $S = \{p, q\}$ with maximum open source side*

$$U = V(B) \setminus (S \cup R_x),$$

as in Section 2.2, where R_x is the component of x in $B - S$. Then neither p nor q can be separated from x by at most two other vertices.

Proof. Suppose that p can be separated from x by at most two other vertices. Since $\mathcal{R}_2(B, x)$ holds, a minimum such separator T has size exactly two. Let Y be the union of the components of $B - T$ that do not contain x, so $p \in Y$.

Consider the separations

$$(A_1, B_1) = (U \cup S,\ V(B) \setminus U),$$
$$(A_2, B_2) = (Y \cup T,\ V(B) \setminus Y).$$

Their join and meet are

$$(A_1 \cup A_2,\ B_1 \cap B_2), \qquad (A_1 \cap A_2,\ B_1 \cup B_2).$$

They are separations. Their orders add up to four, since a vertex-by-vertex count shows that the orders of the join and the meet add up to the orders of the two given separations.

The meet has order at least two.

- If $U \cap Y \neq \varnothing$, the open left side of the meet is nonempty, and x lies on its open right side. A meet of order zero or one would contradict $\mathcal{R}_2(B, x)$.
- If $U \cap Y = \varnothing$, the minimality of S gives a neighbor $h \in U$ of p, since otherwise $\{q\}$ would separate z from x. Since $p \in Y$ and $h \notin Y$, the edge ph forces $h \in T$. The two distinct vertices p and h both belong to the separator of the meet.

Therefore the join has order at most two. Its open source side is $U \cup Y$, which contains z, does not contain x, and strictly contains U, because $p \in Y \setminus U$.

So the separator of the join is a minimum separator between z and x, and its open source side contains $U \cup Y$, which strictly contains U. This contradicts the choice of U in Section 2.2. The argument for q is the same. □

The graph B need not be 2-connected: x may be a cut vertex. Every separator used in the proof excludes x.

9 Repairing one trial contraction with boundedly many searches

Start with a clean instance (K, D). Choose an edge uv of one of these forms:

- the vertex u is not on D, and v is arbitrary;
- the vertices u and v are both on D, and they are consecutive on D.

Contract uv to a new vertex w, suppressing loops and duplicate edges. If $u \notin V(D)$, the cycle is unchanged. If both ends are on D, remove u from the cyclic list and keep the name v for w.

Let H be the contracted graph and $J = H + x$. Set

$$M = N_K(u) \setminus \{v\}.$$

These are the *original markers*. They stay fixed for the whole trial.

9.1 What the contraction guarantees

The old augmented graph $K + x$ satisfied $\mathcal{R}_4$. Therefore:

1. The property $\mathcal{R}_3(J, x)$ holds. A separator of size at most two that contains w lifts to at most three old vertices, namely u, v and the rest of the separator. A separator that avoids w lifts unchanged.
2. Every separator of size at most three that excludes x and disconnects J contains w.
3. For
$$B = J - w,$$
the property $\mathcal{R}_2(B, x)$ holds.

In each case, a separator of J that avoids w also separates $K + x$, because contracting the edge uv does not change whether the rest of the graph is connected. Property 3 follows from property 1, since a separator of B of size at most one, together with w, would separate J.

Furthermore, every component Q of $J - \{w, p, q\}$ that does not contain x, for vertices p, q, contains a marker. Indeed, if Q had no old neighbor of u, then in the old augmented graph all neighbors of Q would lie in $\{v, p, q\}$. That would violate the old $\mathcal{R}_4$.

Each replacement made below changes J and B. Properties 1, 2 and 3 say that certain vertices are joined to x after certain sets of retained vertices are deleted. So, by Lemma 2, they stay true for the current J and $B = J - w$ after every replacement.

9.2 Exact restoration procedure

We process the fixed list M, once for each marker. For a marker z:

1. If z has been deleted, skip it.
2. If z is on the current cycle, skip it. Its edge to x keeps it out of every component that does not contain x.
3. Form $B = J - w$.
4. Run the bounded-flow computation for three internally disjoint paths from z to x in B.
5. If it succeeds, continue with the next marker.
6. Otherwise, obtain the minimum two-cut $\{p, q\}$ with maximum open source side U, as in Section 2.2.
7. Delete all vertices of U from H, and complete the triangle $\{w, p, q\}$.
8. Record one component of $B - \{p, q\}$ inside U, for example the component containing z, as the connected representative for the later lifting.

The failed flow has value exactly two, by $\mathcal{R}_2(B, x)$.

9.3 Why each replacement is valid

Each deleted component A of $B - \{p, q\}$ inside U has exactly the neighborhood
$$N_J(A) = \{w, p, q\}.$$

It has a neighbor at both p and q: otherwise one vertex would separate it from x in B, violating $\mathcal{R}_2(B, x)$. It has a neighbor at w: otherwise $\{p, q\}$ would separate it from x in J, violating $\mathcal{R}_3(J, x)$. It contains no vertex of the current cycle, since every cycle vertex outside $\{w, p, q\}$ is still adjacent to x.

Thus Lemma 1 applies, in its version that deletes several components with a common neighborhood.

9.4 Why no additional markers are needed

We call a retained vertex *good* if no set of at most three other vertices, excluding x, separates it from x. Otherwise it is *bad*. A *bad component* is a component, not containing x, of the graph obtained from J by deleting at most three vertices other than x. All its vertices are bad.

At every replacement made during restoration, the three boundary vertices w, p, q are good.

- The vertex w is good, because by property 2 of Section 9.1 every separator of size at most three contains w.
- By Lemma 6, applied to B, the vertices p and q cannot be separated from x in B by at most two vertices. Since every separator of J of size at most three contains w, they are good in J.

Goodness stays true under later replacements, by Lemma 2.

Now consider a bad component after such a replacement. It cannot contain any of the good boundary vertices. Therefore it contains no end of a newly added clique edge, and it has no old edge into the deleted part, whose neighbors are w, p, q. So it was already *the same component, with the same vertex set*, before the replacement.

Consequently it keeps its old marker. This proves, by induction, that every bad component during restoration contains an original marker that is still present. At the start of restoration this holds by the last statement of Section 9.1, since by properties 1 and 2 every bad component comes from a separator $\{w, p, q\}$. After the marker list is exhausted:

- a remaining marker whose flow succeeded is good, because a separator of J of size at most three would contain w, and the rest of it would separate z from x in B;
- a marker whose flow failed was deleted;
- a skipped cycle marker is good, because it is adjacent to x.

A remaining bad component would therefore contain a good vertex, which is a contradiction. So no bad component remains, and the instance is clean.

Lemma 7. *Restoration returns a clean instance that has a crossing if and only if the contracted instance has one, and it uses at most $|M|$ flow queries for three paths.*

The bound holds even for a trial that is rejected in the end. There is no hidden search over all remaining vertices.

10 The complete contraction algorithm

After the initial cleaning, call Classify.

- On `NO`, report that C is crossless.
- On `WITNESS`, lift the crossing through the recorded initial replacements and return it.
- On `YES`, the current graph is clean, nonplanar and crossed.

We repeat the following iteration while in the last state.

One iteration.

1. *Sparsify* as in Section 7.2, if needed. The graph stays clean, nonplanar and crossed, and now $M \leq 16N$.
2. Choose any five distinct vertices of degree at most 64.
3. For each chosen vertex u:
   - if $u \notin V(D)$, try every incident edge uv;

- if $u \in V(D)$, try its two cycle edges, except that contractions of cycle edges are skipped when $|D| = 4$.

4. For each trial:
   - copy the current instance;
   - contract uv;
   - restore cleanliness as in Section 9, using the markers $N_K(u) \setminus \{v\}$;
   - call Classify on the restored instance.
5. On `NO`, discard that trial.
6. On `WITNESS`, lift the crossing through the replacements and the contraction of that trial, then through all earlier accepted operations, and return it.
7. On `YES`, accept that trial and begin the next iteration.

All choices in the graph and all orders of adjacency lists may be fixed arbitrarily.

10.1 Five vertices of low degree always exist

A nonplanar graph has at least five vertices. For $5 \leq N \leq 9$, every vertex has degree at most eight. For $N \geq 10$, the number of vertices of degree at least 65 is at most

$$\frac{2M}{65} \leq \frac{32N}{65} < \frac{N}{2}.$$

Thus more than $N/2 \geq 5$ vertices have degree at most 64. No exhaustive base case for small instances is needed.

10.2 At least one trial is positive

Fix any crossing of D. It has exactly four ends. So at least one of the five chosen vertices, say u, is not one of these ends. There are three cases.

- *The vertex u is off the cycle and lies on a crossing path.* Contract an edge of that path at u. The crossing survives.
- *The vertex u is off the cycle and lies on neither path.* Contract any edge at u. The crossing survives. Such an edge exists, because cleanliness implies that every vertex off the cycle has degree at least four.
- *The vertex u is on the cycle.* Then neither crossing path uses it. Contract either cycle edge at u. The cyclic order of the four ends of the crossing does not change. Here $|D| \geq 5$, since D contains the four ends and u, so this trial is not skipped.

Restoration preserves the existence of a crossing, by Lemma 1. Since the classification is exact, it accepts at least one trial.

Every accepted trial contracts an edge and so decreases the number of vertices. Thus the procedure terminates. It cannot end in a nonplanar positive state with fewer than five vertices, so before that happens the classification returns a crossing.

11 Lifting contractions

For completeness, here are all the lifts through contractions.

The record stores the graph and the cycle before the contraction, and the edge uv. At most one returned crossing path uses the new vertex w, since the two paths are disjoint.

Both ends off the cycle. If w is unused, keep the paths. Otherwise let a and b be the vertices before and after w on its path. In the recorded graph, choose

$$u_a \in \{u, v\} \text{ adjacent to } a, \qquad u_b \in \{u, v\} \text{ adjacent to } b.$$

If $u_a = u_b$, replace w by that vertex. Otherwise replace it by u_a, u_b, using the edge uv. The added vertices do not occur on the other path or elsewhere on this path.

One end on the cycle. The algorithm orients this case so that $u \notin V(D)$ and $v \in V(D)$, and keeps $w = v$. If w is used, it is an end of a crossing path. Suppose that the path starts at w. If its first edge comes from v in the recorded graph, keep the end v. If it comes from u, replace the beginning by v, u, followed by that edge. For a path that ends at w, reverse the order. The only vertex added inside the path is u, which is off the cycle.

Both ends on the cycle. Then u and v were consecutive on the recorded cycle. If w is used, it is an end of a crossing path. Choose whichever of u, v is adjacent, in the recorded graph, to the next vertex of the path, and use it as the end. *Do not insert the cycle edge uv into the path.* Replacing one cycle position by one of two consecutive positions keeps the cyclic order of the four distinct ends, since no other end lies between those two positions.

Thus every contraction lifts without a search. Reversing all accepted operations gives two disjoint C-paths in the original graph, with alternating ends.

12 Search count and total running time

Let n_0 and m_0 be the numbers of vertices and edges of the component that contains the original cycle. This component is connected and contains a cycle, so $m_0 \geq n_0$.

Operation	Number
Initial flow queries for four paths	at most n_0
Accepted contraction iterations	at most n_0
Trials per iteration	at most $5 \cdot 64 = 320$
Flow queries for three paths per trial	at most 63
BFS forests per sparsification	four
Boolean planarity tests	at most $320n_0 + 1$
Searches constructing the final bridge crossing	two
Searches reversing separator replacements	at most n_0

A flow query for three paths uses at most three residual searches. On failure, finding the maximum cut side and the relevant components takes a bounded number of further linear traversals. Thus the total number of residual and ordinary graph searches is $O(n_0)$.

The planarity tests are *separate linear-time calls.* We do not count them as ordinary path searches.

Every accepted separator replacement deletes at least one vertex and adds at most three edges. Contractions add no edges before simplification. Temporary trials make only a bounded number of replacements. Consequently every graph that is searched has size

$$O(n_0 + m_0).$$

All remaining work is linear per bounded-flow query or per trial: graph copies, contraction and simplification, bridge construction, counting sorts, stack operations, certificate construction, and

path splicing. Storing complete snapshots is allowed, since the stack of accepted operations contains only $O(n_0)$ operations.

Therefore the total time is

$$O(n_0(n_0 + m_0)) = O(n_0 m_0) \subseteq O(nm).$$

13 Assumptions, critical choices, and tests

The algorithm runs on the given pair (G, C). It assumes only that G is finite and simple and that C is a cycle of G.

The following choices matter:

- BFS (scan-first) forests for the certificate. Arbitrary spanning forests do not suffice.
- A minimum two-cut with *maximum open source side* during restoration.
- Contractions only of edges at a vertex off the cycle, or of edges between consecutive cycle vertices.

The file `crossing_algorithm.py` in this folder is a reference implementation in Python. It has the interface

```
from crossing_algorithm import find_crossing

result = find_crossing(n, edges, cycle)
# None, or (path1, path2)
```

It uses the planarity test of the companion note [HJ26], from the file `planarity_linear.py` in the folder `docs/planarity`, and no other library outside the Python standard library. The default call follows the algorithm above. After sparsifying, it also calls BridgeCross once more, which is not in the algorithm above and does not change the bounds. The optional mode `audit=True` checks cleanliness again after every trial. This mode is for testing, and it does not meet the bound $O(nm)$. Python sets and dictionaries are not the arrays of a word RAM, so the implementation illustrates the algorithm but does not prove its time bound.

We tested the implementation as follows. On 9,000 random instances with at most 19 vertices, the answer agreed with an exhaustive search for a crossing, with the audit mode on. These instances include 3,731 runs of the contraction loop with 15,296 trials, many of them with restoration replacements. On 40 dense random instances with 36 to 45 vertices, the sparsification step ran 35 times. Every returned crossing is checked against the original graph before it is returned. Separately, on 100,000 random instances, BridgeCross agreed with a direct check for alternating attachments of two distinct bridges. These tests check the implementation. The correctness and the time bound follow from the lemmas above.

14 The Lean formalization

The Lean formalization of this note is in the repository `GraphMinor`.

- The folder `GraphMinor/Crossing` proves the mathematics of Sections 1 to 11, apart from Fact P. The file `GraphMinor/Crossing/Statements.lean` states the claims of Sections 2, 3, 4, 7, 8, 9, 10 and 11 as propositions, each with the passage it formalizes. Fact T is proved in `GraphMinor/Crossing/PlanarFactT.lean` from the formalization of Humeau and Pous in the folder `GraphMinor/HP25`.

- In the folder `GraphMinor/Compute`, the files `Crossing*.lean` and `SpecsCrossing*.lean` give word-RAM programs for all steps and prove their correctness and running times. The whole algorithm is `CrossingExtractorStatement`, proved as `crossingExtractor_of`.
- The file `GraphMinor/Compute/SpecsCrossingReductionProved.lean` derives from it the two paths statement `Shiloach80Statement` used in (8.1), by the reduction recalled in the introduction. The file `GraphMinor/Compute/DisjointPathsFinal.lean` proves the main result of Graph Minors XIII, `disjointPathsStatement_proved`, with no hypotheses.

The Lean formalization differs from this note in the following places.

- *The model of computation.* The note counts searches and argues that the rest of the work is linear per search. The Lean proves executions of fixed word-RAM programs, with explicit word sizes, memory layouts and record buffers, and bounds their numbers of steps. A search count alone does not show that a program with bounded words meets the time bound.
- *The planarity test.* Fact P is the Lean statement `PlanarityTestStatement`. It is not assumed: it is proved, as `planarityTestStatement_proved`, from the companion note [HJ26].
- *The use in Graph Minors XIII.* The algorithm of this note returns paths. The Lean uses it for the step of (8.1), and then assembles the algorithm for the main result of Graph Minors XIII, which decides the disjoint paths problem for a fixed number of terminal pairs. That final program returns a Boolean answer, as the problem is stated in Graph Minors XIII, page 65. It is not required to return the paths.

References

[CKT93] J. Cheriyan, M.-Y. Kao and R. Thurimella, Scan-first search and sparse certificates: an improved parallel algorithm for k-vertex connectivity, *SIAM J. Comput.* **22** (1993), 157–174.

[GM13] N. Robertson and P. D. Seymour, Graph Minors. XIII. The disjoint paths problem, *J. Combin. Theory Ser. B* **63** (1995), 65–110.

[HJ26] M. Hajiaghayi and M. JafariRaviz, A Boolean linear-time planarity test, note, November 2026, file `docs/planarity/planarity_test.tex` of the repository `GraphMinor`.

[HP25] S. Humeau and D. Pous, On the two paths theorem and the two disjoint paths problem, arXiv:2505.16431v2 (14 August 2025).

[HT74] J. Hopcroft and R. Tarjan, Efficient planarity testing, *J. ACM* **21** (1974), 549–568.

[Sh80] Y. Shiloach, A polynomial solution to the undirected two paths problem, *J. ACM* **27** (1980), 445–456.